

Summer Internship Report on

TOXIGUARD: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted By

Anumathi Singh

Roll no: 160125862006

Under the Esteemed Internal Guidance of

Dr. B. Indira

Associate Professor

Department of MCA, CBIT (A)



Department of MCA

Chaitanya Bharathi Institute of Technology (Autonomous)

Gandipet, Hyderabad — Telangana

Academic Year: 2026–27



CERTIFICATE

This is to certify that the Internship Report titled “ToxiGuard: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection” is a bona fide record of work carried out by Anumathi Singh (Hall Ticket No. 160125862006), in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA).

The report presents the ToxiGuard project as part of the summer internship requirement following the Artificial Intelligence program at EduExpose.

[Internship period: 04-07-2026 – 15-08-2026]

Internship Internal Guide

Head of the Department

Dr. B. Indira

Dr. B. Indira

Associate Professor

Associate Professor

Dept. of MCA, CBIT (A), Hyderabad

Dept. of MCA, CBIT (A), Hyderabad

ORGANIZATION CERTIFICATE

The completion certificate issued by EduExpose for the Artificial Intelligence internship program is reproduced below.



DECLARATION

I, Anumathi Singh, bearing Hall Ticket No. 160125862006, hereby declare that the internship project report entitled “ToxiGuard: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection”, submitted as part of the requirements of the Master of Computer Applications (MCA) program, is an original work carried out by me under the guidance of my internal guide, Dr. B. Indira.

I further declare that this report presents the design, implementation, testing, and documentation of the ToxiGuard toxic-comment detection system. The report has not been previously submitted, either in whole or in part, to any institution or university for the award of any degree, diploma, certification, or academic credit.

All tools, libraries, datasets, and technical references used in the preparation of this report have been acknowledged to the best of my knowledge and belief.

Anumathi Singh

Hall Ticket No: 160125862006

Place: Hyderabad

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me in completing my internship project titled “ToxiGuard: An NLP Based Machine Learning Framework for Cyberbullying and Toxic Comment Detection”.

I am deeply grateful to Dr. B. Indira, my Internal Guide and Head of the MCA Department at Chaitanya Bharathi Institute of Technology (CBIT), for her guidance, continuous encouragement, and valuable suggestions throughout the project.

I also thank Dr. C. V. Narasimhulu, Principal, Chaitanya Bharathi Institute of Technology, for his support in this endeavor.

I extend my heartfelt thanks to EduExpose for the guidance and learning support provided during the Artificial Intelligence internship program.

Finally, I thank the faculty members of the Department of MCA, and my family and friends, for their constant encouragement and support throughout the project.

Anumathi Singh

TABLE OF CONTENTS

1. Introduction.....	11
1.1 Introduction to the Internship.....	11
1.2 Project Background.....	11
1.3 Objectives and Scope.....	11
2. Organization Profile.....	12
2.1 Internship Program.....	12
2.2 Academic Setting	12
2.3 Relevance of the Program.....	12
3. Internship Objectives and Activities	13
3.1 Technical Objectives.....	13
3.2 Activities Reflected in the Project	13
3.3 Deliverables	13
4. Problem Statement.....	14
4.1 Existing Approach	14
4.2 Proposed System.....	14
4.3 Boundaries	14
5. Technologies and Tools Used.....	15
6. System Analysis.....	16
6.1 Functional Requirements	16
6.2 Nonfunctional Requirements	16
6.3 Hardware and Software.....	16
7. System Design	17
7.1 Data Flow Diagram.....	17
7.2 Entity Relationship Diagram.....	19
8. Implementation	20

8.1 Modules.....	20
8.2 Data Preparation.....	20
8.3 Feature Extraction and Training	20
8.4 Selection and Serialization.....	20
8.5 Streamlit Application	21
9. Testing.....	22
9.1 Test Strategy	22
9.2 Functional Checks.....	22
9.3 Interpretation.....	22
10. Results and Discussion	23
10.1 Dataset and Experiment	23
10.2 Model Comparison.....	24
10.3 Application Output.....	27
10.4 Limitations	28
11. Learning Outcomes	29
12. Challenges and Solutions.....	30
12.1 Class Imbalance	30
12.2 Training and Inference Consistency	30
12.3 Interpretation and Context	30
13. Conclusion	31
14. Future Scope	32
REFERENCES	33
Appendix A — Source Code	34
Appendix B — Paper Presentation Certificate	38
Appendix C — Project Outputs	39

LIST OF FIGURES

Fig No.	Title	Page No.
Fig 7.1	DFD Level 0 — ToxiGuard Context	17
Fig 7.2	DFD Level 1 — Training and Prediction	18
Fig 7.3	Logical Entity Relationship Model	19
Fig 8.1	Streamlit Application Overview	21
Fig 10.1	Exploratory Data Analysis	23
Fig 10.2	Comparison of Classifier Performance	25
Fig 10.3	Logistic Regression Confusion Matrix	26
Fig 10.4	Single-Comment Prediction Output	27

LIST OF TABLES

Table No.	Title	Page No.
Table 5.1	Technologies and Tools Used	15
Table 6.1	Functional Requirements	16
Table 8.1	Main Project Modules	20
Table 9.1	Functional Test Cases	22
Table 10.1	Model Comparison	24
Table 10.2	Sample Prediction Results	28
Table C.1	Project Output Files	39

ABSTRACT

ToxiGuard is a machine learning prototype that identifies potentially toxic online comments for human review. The project uses the Jigsaw toxic-comment dataset and treats the published toxic label as a binary target. After missing and duplicate comments were removed and text was cleaned, 159,556 comments remained. A stratified sample of 30,000 comments was used for the reported experiment, with 6,000 comments reserved for testing.

The system converts cleaned text into 15,000 TF-IDF unigram and bigram features. Six supervised classifiers are compared on the same held-out split: Multinomial Naive Bayes, Complement Naive Bayes, Logistic Regression, calibrated Linear SVM, SGD Classifier, and Random Forest. The selection rule prioritizes the toxic-class F1 score, then recall and accuracy. Logistic Regression ranked first, with 93.23% accuracy, 61.02% toxic precision, 81.39% toxic recall, 69.75% toxic F1, and 0.9576 ROC-AUC.

The selected model and vectorizer are saved and loaded by a Streamlit application for single-comment and batch predictions. The interface presents a predicted class and toxic probability, while making clear that a moderator should inspect context before acting. The report covers the data preparation, design, implementation, testing, results, limitations, and future improvements of the project.

1. Introduction

1.1 Introduction to the Internship

The MCA summer internship offers a setting in which classroom knowledge can be applied to data handling, model evaluation, and software delivery. The EduExpose certificate confirms completion of an Artificial Intelligence program from 4 July to 15 August 2026. ToxiGuard is the practical project documented in this report.

1.2 Project Background

Online discussions can contain insults and other harmful language at a scale that is difficult to review manually. ToxiGuard predicts whether a comment resembles toxic examples in a labelled dataset. It provides a review signal and a probability estimate; final moderation decisions require human interpretation of the surrounding conversation.

1.3 Objectives and Scope

- Prepare a labelled English toxic-comment dataset and examine class balance.
- Compare multiple conventional classifiers using the same TF-IDF representation and test split.
- Deploy the best measured model in a Streamlit interface for single and batch analysis.
- Document the performance and limits of the resulting system.

The current scope is English-language binary classification. It does not classify the separate Jigsaw categories in the user interface, use conversation history, or automate content removal.

2. Organization Profile

2.1 Internship Program

EduExpose provided the Artificial Intelligence program completed by Anumathi Singh from 4 July 2026 to 15 August 2026. The program forms the internship context for this academic submission. ToxiGuard applies machine learning concepts to the practical problem of identifying toxic online comments.

2.2 Academic Setting

The work is submitted to the Department of MCA, Chaitanya Bharathi Institute of Technology (Autonomous), Gandipet, Hyderabad. Dr. B. Indira serves as both the Internal Guide and Head of Department for this submission. The project connects natural language processing, experimental model comparison, and a working web interface.

2.3 Relevance of the Program

The Artificial Intelligence program is directly relevant to text classification. ToxiGuard demonstrates data cleaning, feature extraction, supervised learning, metric selection, and deployment. The report emphasizes what is present in the project repository and what the measured outputs actually show.

3. Internship Objectives and Activities

3.1 Technical Objectives

The project objectives were to assemble a reproducible training pipeline, choose a classifier from measured toxic-class performance, and make predictions accessible through a simple dashboard. The experiment uses a fixed random seed, stratified sampling, and one shared hold-out set for all six candidates.

3.2 Activities Reflected in the Project

- Examining the source CSV and removing missing, duplicate, and empty comments.
- Implementing shared text cleaning in `src/text_utils.py`.
- Generating exploratory plots and TF-IDF features.
- Training six candidates and exporting comparison metrics, classification report, and confusion matrix.
- Loading the selected model and vectorizer into the Streamlit dashboard.
- Running sample predictions and documenting the outputs and limitations.

3.3 Deliverables

The project directory contains source code, the cleaned dataset, saved model and vectorizer files, a training notebook, figures, prediction exports, and the Streamlit application. This report collects the evidence needed to understand and review those deliverables.

4. Problem Statement

4.1 Existing Approach

Manual moderation requires reviewers to inspect individual comments. This provides context, but becomes time consuming as comment volume grows. A simple keyword list is quick but cannot reliably handle variation in wording, negation, or benign uses of flagged terms.

4.2 Proposed System

ToxiGuard applies the same preprocessing used during training to a new comment, transforms it with the saved TF-IDF vectorizer, and produces a toxic-class probability from the selected Logistic Regression classifier. The dashboard displays the prediction, probability, and review priority. It also supports CSV-based batch analysis.

4.3 Boundaries

The model learns from labelled Wikipedia comments rather than every social platform. Sarcasm, quotations, identity terms, code switching, and changing slang can produce errors. The reported test metrics describe performance on the selected held-out dataset, not guaranteed performance in deployment.

5. Technologies and Tools Used

Table 5.1: Technologies and Tools Used

Component	Role in ToxiGuard
Python	Training script, preprocessing, and dashboard logic
pandas and NumPy	Tabular cleaning, analysis, and prediction export
scikit-learn	TF-IDF, train/test split, classifiers, and metrics
matplotlib and seaborn	EDA, comparison chart, and confusion matrix
joblib	Saving and loading the selected model and vectorizer
Streamlit	Interactive single and batch comment analysis
Jigsaw CSV	Labelled English comment source

The project uses files rather than a relational database. The source and cleaned records are CSV files; trained artifacts are serialized as PKL files. The application reads these outputs from the local project directory.

6. System Analysis

6.1 Functional Requirements

Table 6.1: Functional Requirements

Requirement	Implementation
Input	Accept one comment or a CSV batch in the dashboard
Preparation	Apply the same cleaning function to training and new text
Prediction	Return a binary class and toxic probability
Evaluation	Display model comparison and confusion matrix
Export	Save test predictions and allow batch download

6.2 Nonfunctional Requirements

The interface should be legible and usable on a normal laptop. Training results should be reproducible from the documented CSV and fixed random seed. The application should load the same vectorizer and selected model that were saved by training. Predictions are decision support and should be accompanied by a human-review warning.

6.3 Hardware and Software

A standard Windows laptop with Python 3 and enough free storage for the CSV, model files, and generated charts is sufficient. The project requirements list Streamlit, pandas, NumPy, scikit-learn, joblib, matplotlib, seaborn, Jupyter, and ReportLab. The dashboard is launched with `python -m streamlit run app.py` from the ToxiGuard directory.

7. System Design

7.1 Data Flow Diagram

The context diagram shows the main inputs and outputs of the system. The level 1 diagram separates the training path from prediction. Both paths use the same cleaning rules; the prediction path loads the saved vectorizer and classifier.

Fig 7.1: DFD Level 0 — ToxiGuard Context

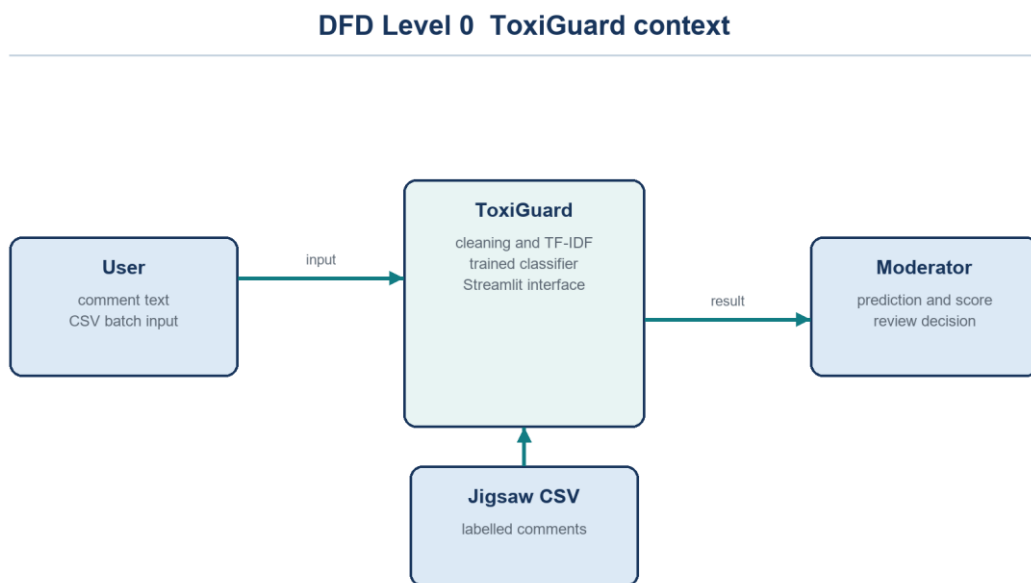


Fig 7.1 represents ToxiGuard as a single process. The user supplies a comment or a batch of comments, while the Jigsaw dataset supplies labelled training examples. The system returns a predicted class and probability to support a moderator's review.

Fig 7.2: DFD Level 1 — Training and Prediction

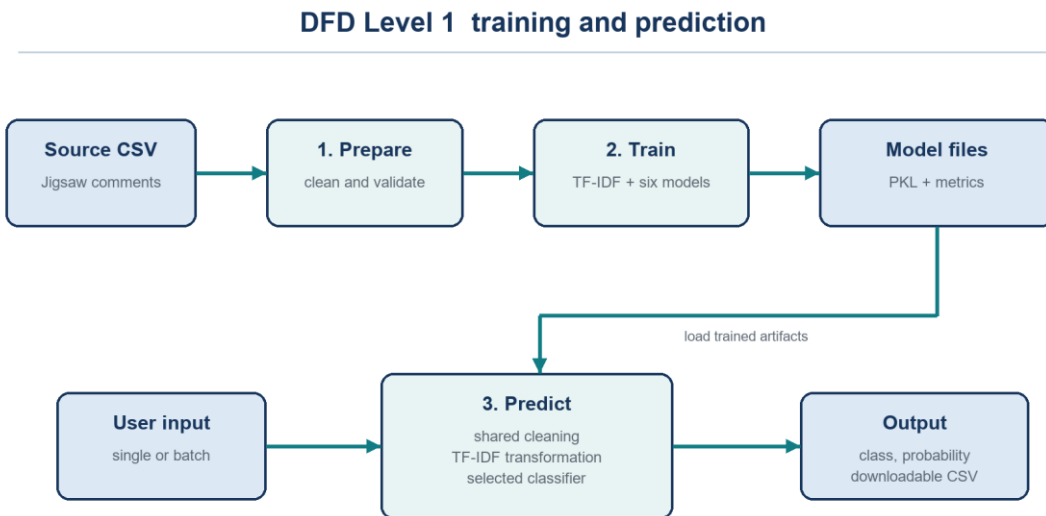


Fig 7.2 separates data preparation, model training, and prediction. The training path cleans the source comments, creates TF-IDF features, compares six classifiers, and saves the selected model. The prediction path applies the same cleaning rules to user input and loads the saved artifacts to generate a class and probability.

7.2 Entity Relationship Diagram

The ER diagram is a logical description of the data and evaluation records. It does not imply a database implementation. The project stores comments and predictions as CSV files and model artifacts as PKL files.

Fig 7.3: Logical Entity Relationship Model

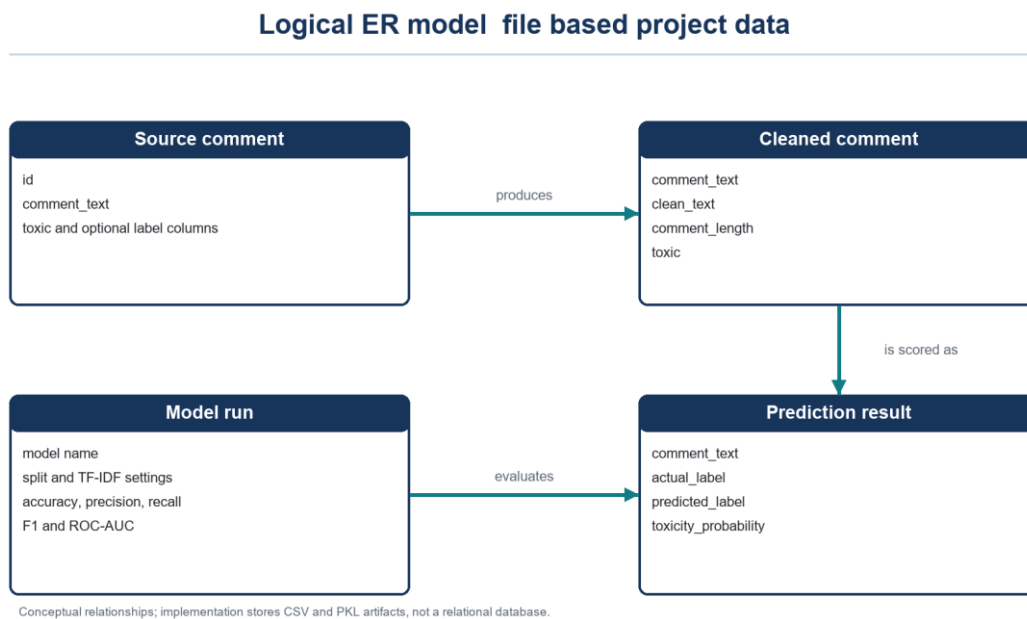


Fig 7.3 relates source comments, cleaned comments, model runs, and prediction results. Cleaning creates the text representation used by the classifier. A model run records evaluation metrics, and prediction records contain the actual label, predicted label, and toxic probability. These are conceptual entities represented by the project's files.

8. Implementation

8.1 Modules

Table 8.1: Main Project Modules

Module	Purpose
download_dataset.py	Obtains the public source CSV
src/text_utils.py	Provides text cleaning and label helpers
train_model.py	Cleans, trains, compares, and exports outputs
app.py	Loads saved artifacts and presents predictions

8.2 Data Preparation

The training script reads `comment_text` and `toxic`, removes rows with missing required values and duplicate comment text, applies the shared cleaning function, and removes records whose cleaned text is too short. It records comment length for EDA. The reported cleaned dataset contains 159,556 rows, including 15,293 toxic comments and 144,263 non-toxic comments.

8.3 Feature Extraction and Training

A stratified sample of 30,000 rows is split 80:20 with random state 42. The TF-IDF vectorizer is fitted on training text only and transforms the test text without refitting. It uses unigrams and bigrams, `min_df=2`, sublinear term frequency, and a 15,000-feature cap. Six classifiers receive the same feature matrix.

8.4 Selection and Serialization

Each model is assessed with accuracy, toxic-class precision, recall, F1, and ROC-AUC. The comparison is sorted by toxic-class F1, then recall, then accuracy. The selected Logistic Regression model and TF-IDF vectorizer are saved separately with `joblib`. The output folder holds the comparison table, confusion matrix, classification report, prediction export, EDA plots, and metadata.

8.5 Streamlit Application

The dashboard offers Overview, Analyze comment, Batch analysis, Model performance, Dataset insights, and About the project pages. Its Analyze comment page takes a text input and displays a class, toxic probability, model confidence, and review priority. The Batch analysis page accepts comments in a CSV and exports predicted results.

Fig 8.1: Streamlit Application Overview

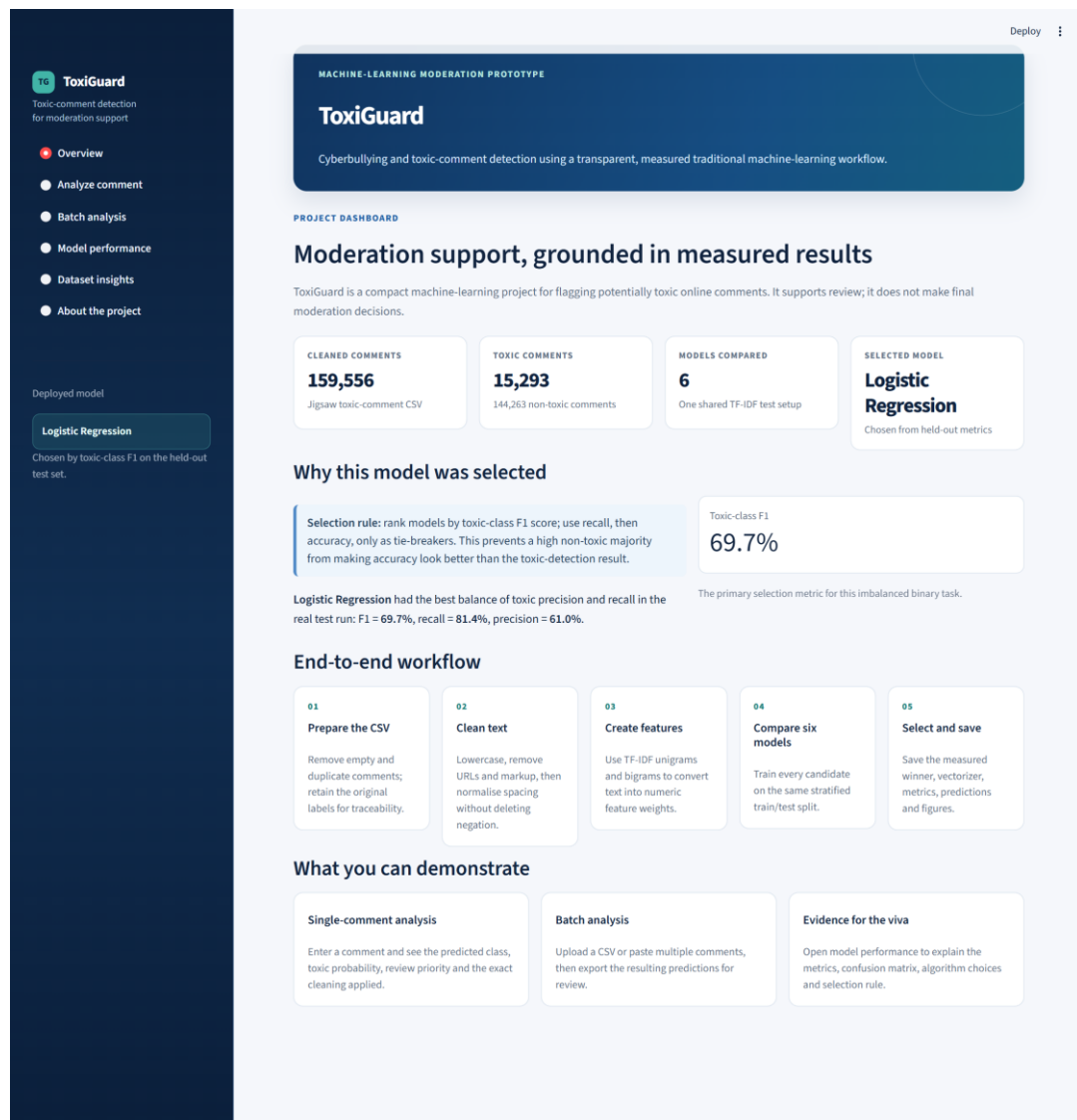


Fig 8.1 shows the dashboard overview, including the cleaned dataset size, toxic-comment count, six-model comparison, and selected Logistic Regression model. The sidebar provides access to comment analysis, batch analysis, model performance, and dataset insights.

9. Testing

9.1 Test Strategy

The training script was run against the local Jigsaw CSV and completed with six measured model results. The dashboard was started locally and its Overview, Analyze comment, and Model performance pages were inspected. Sample comments were evaluated through the saved artifacts using the same cleaning and vectorization path.

9.2 Functional Checks

Table 9.1: Functional Test Cases

ID	Input or action	Observed result
T01	Start Streamlit dashboard	Application loaded with Logistic Regression selected
T02	Insulting sample comment	Toxic, probability 0.9915
T03	Appreciative sample comment	Non-toxic, probability 0.1427
T04	Polite disagreement	Non-toxic, probability 0.0501
T05	Open model performance	Six-model comparison and selected metrics displayed

9.3 Interpretation

These checks confirm that the saved artifacts load and the interface produces a result for representative input. Three sample comments are not a statistical validation set; the 6,000-row held-out evaluation provides the quantitative evidence. A future acceptance test should include human-reviewed comments from the intended deployment setting.

10. Results and Discussion

10.1 Dataset and Experiment

The cleaned dataset contains 159,556 comments, with the toxic class representing about 9.6% of rows. A stratified 30,000-row subset was divided into 24,000 training comments and 6,000 test comments. This imbalance is why accuracy alone is insufficient for model selection.

Fig 10.1: Exploratory Data Analysis

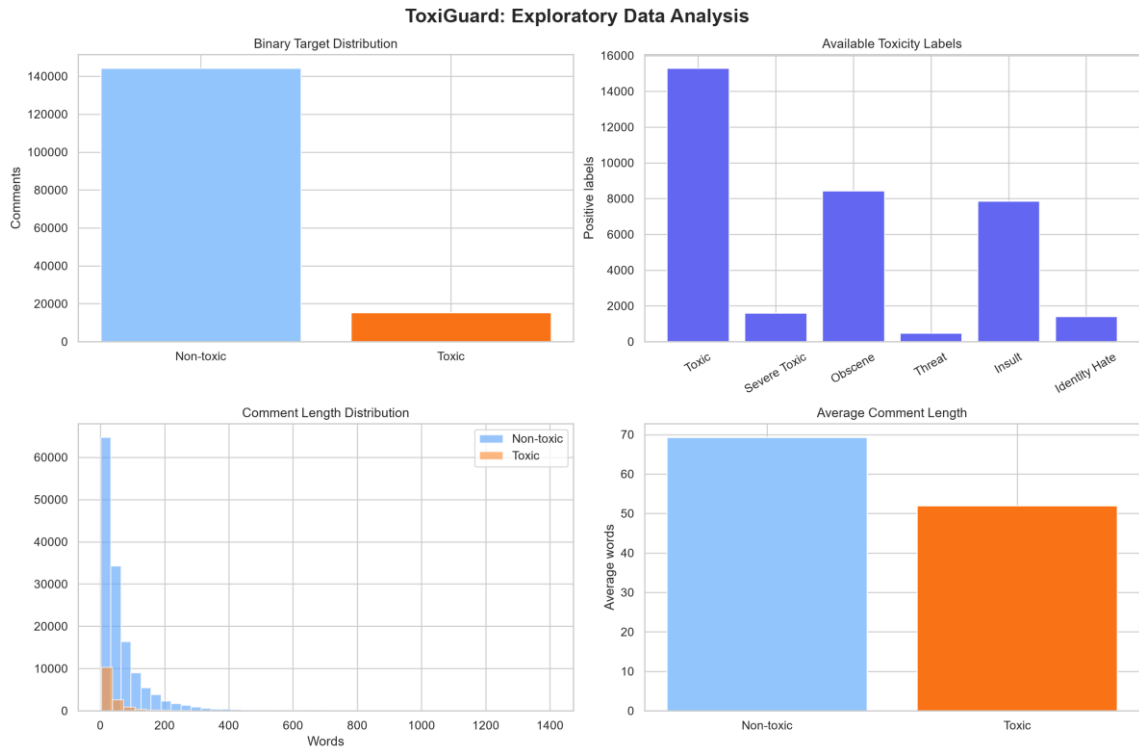


Fig 10.1 shows four views of the cleaned dataset: binary class counts, counts for the available toxicity labels, comment-length distributions, and average comment length. Non-toxic comments greatly outnumber toxic comments. The skewed length distribution and unequal class sizes provide context for preprocessing and minority-class evaluation.

10.2 Model Comparison

Table 10.1: Model Comparison

Model	Acc	Prec	Recall	F1	AUC
Logistic Regression	0.932	0.610	0.814	0.697	0.958
SGD Classifier	0.932	0.613	0.795	0.692	0.953
Linear SVM (Calibrated)	0.950	0.855	0.574	0.687	0.956
Multinomial Naive Bayes	0.945	0.926	0.459	0.614	0.945
Complement Naive Bayes	0.892	0.467	0.864	0.606	0.945
Random Forest	0.917	0.558	0.628	0.591	0.914

Table 10.1 reports held-out accuracy (Acc), toxic precision (Prec), toxic recall, toxic F1, and ROC-AUC (AUC). Logistic Regression achieved the highest toxic-class F1 of 0.6975. It recovered about 81.4% of toxic comments at a toxic precision of about 61.0%. The calibrated Linear SVM had higher overall accuracy and precision, but lower toxic recall. The choice therefore reflects the documented F1-first selection rule.

Fig 10.2: Comparison of Classifier Performance

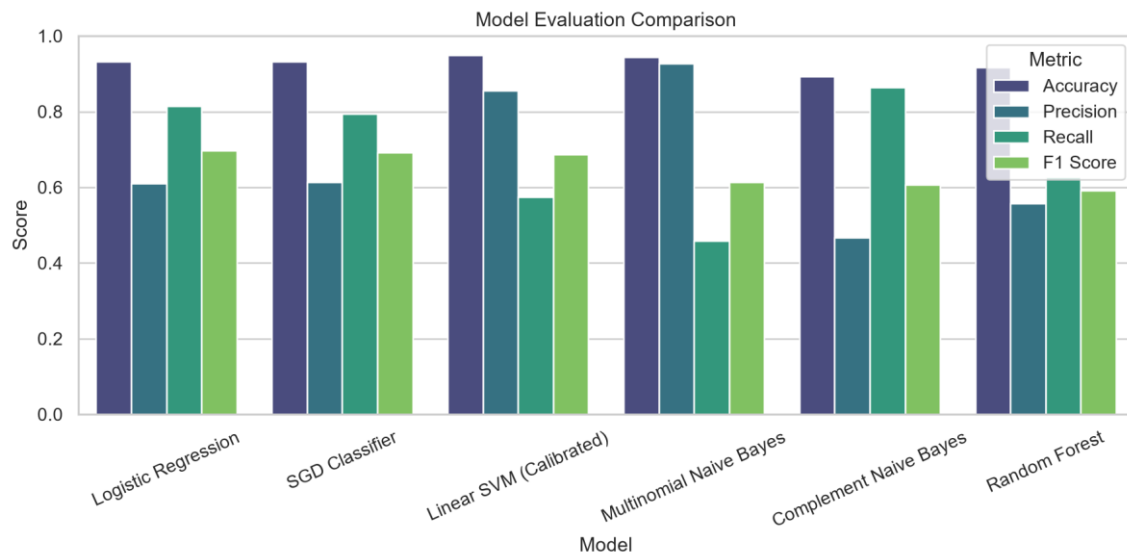


Fig 10.2 compares accuracy, toxic precision, toxic recall, and toxic F1 for the six classifiers. Multinomial Naive Bayes has high precision but misses more toxic comments. Logistic Regression provides the highest toxic F1, while Complement Naive Bayes achieves higher recall at the cost of more false positives.

Fig 10.3: Logistic Regression Confusion Matrix

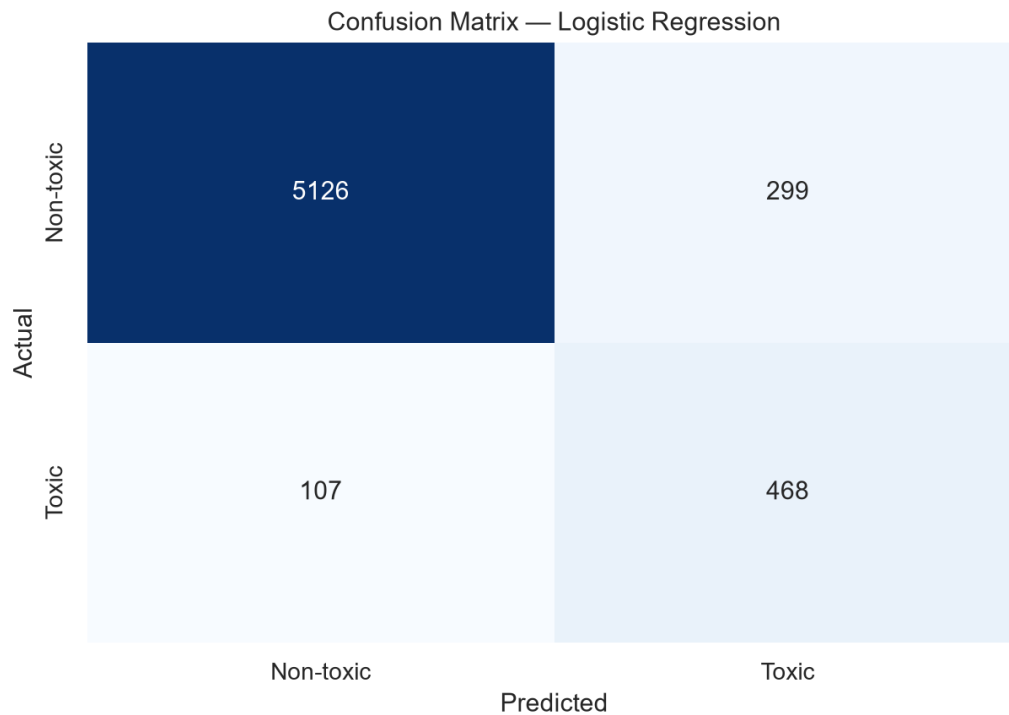


Fig 10.3 summarizes predictions for 6,000 test comments. The classifier correctly identifies 5,126 non-toxic comments and 468 toxic comments. It incorrectly flags 299 non-toxic comments and misses 107 toxic comments. These errors explain why automated flags still require human review.

10.3 Application Output

The running Streamlit application loaded Logistic Regression from the saved model files. For the test sentence “You are an idiot and nobody wants you here,” it displayed a Toxic prediction with a toxic probability of about 99.1%. This example demonstrates the interface output, not the accuracy of the model on the wider population.

Fig 10.4: Single-Comment Prediction Output

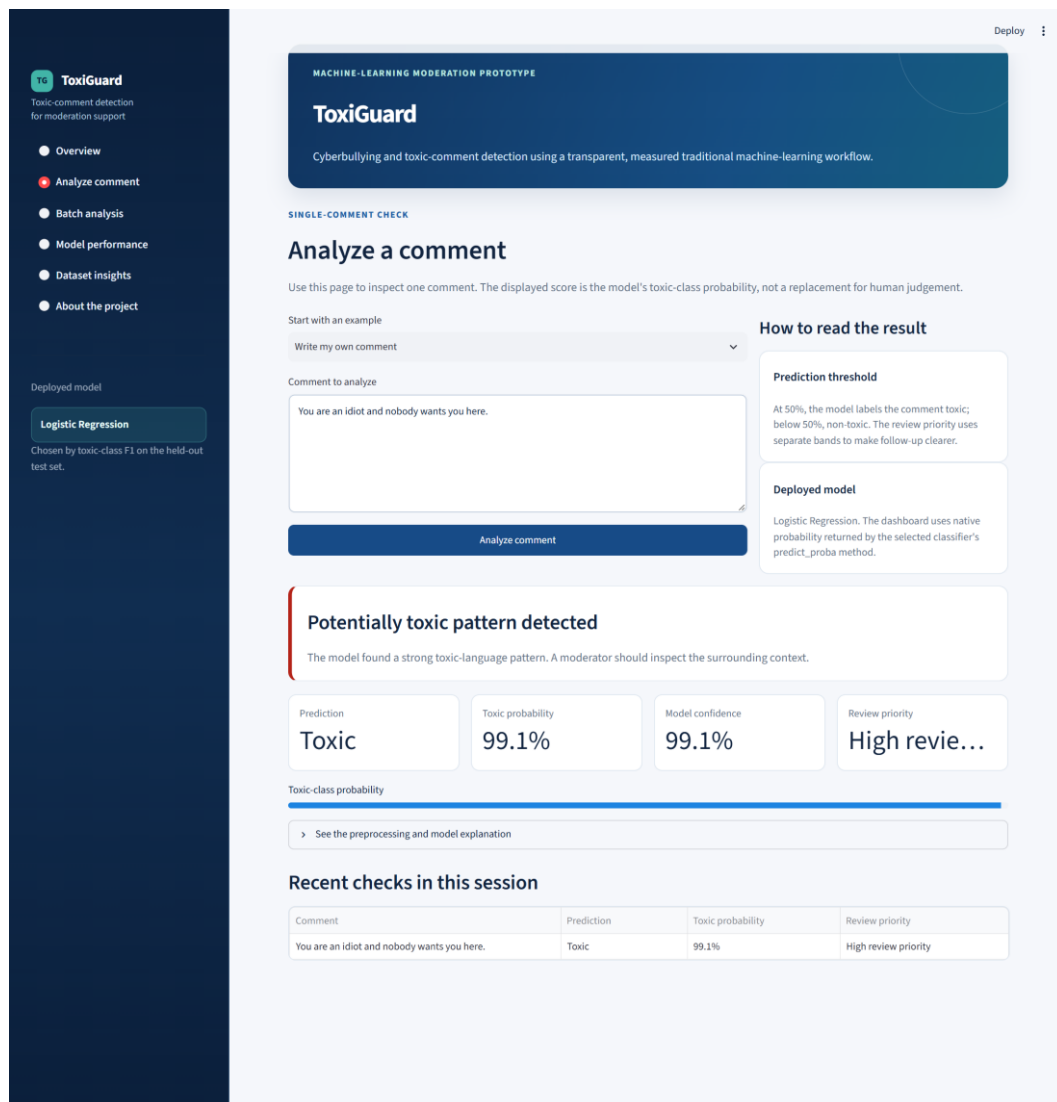


Fig 10.4 shows the entered comment, Toxic prediction, 99.1% toxic probability, and session history. The result cards communicate the model output; the review message prompts the user to consider the surrounding context before making a moderation decision.

Table 10.2: Sample Prediction Results

Sample	Toxic probability	Predicted class
Insulting sample	0.9915	Toxic
Appreciative sample	0.1427	Non-toxic
Polite disagreement	0.0501	Non-toxic

10.4 Limitations

The measured metrics arise from the held-out Jigsaw subset. The model can miss toxic comments and flag benign comments; precision 0.6102 means many flags still require review. Dataset bias, unfamiliar languages, obfuscation, quotations, and conversation context are not resolved by this binary TF-IDF model. The confidence value is a model estimate and should not be treated as a guarantee.

The same test split was used to compare and select the six models. The reported scores are therefore comparative experiment results; an independent final test set or nested validation is needed for a less biased estimate after model selection.

11. Learning Outcomes

The project required a complete path from raw text to a working application. It reinforced the need to inspect class balance, fit text features only on training data, compare models using metrics that reflect the minority class, and keep preprocessing consistent between training and inference.

The report and dashboard also required presenting technical results clearly. A reviewer can see the selected model, its measured tradeoffs, the confusion matrix, and examples of the interface. The paper-presentation certificate included in Appendix B records an additional opportunity to communicate related work to an academic audience.

12. Challenges and Solutions

12.1 Class Imbalance

Only 15,293 of the 159,556 cleaned comments carry the toxic label. The split and experiment subset were stratified to retain the class ratio. The selection rule uses toxic F1 first, which reduces the chance of choosing a model solely because it predicts the majority class well.

12.2 Training and Inference Consistency

A different cleaning path in the dashboard would change the feature values presented to the trained model. ToxiGuard imports the same `clean_text` helper in training and inference and loads the exact vectorizer saved by training.

12.3 Interpretation and Context

Text classification cannot reliably infer intent or the full conversation context from one comment. The dashboard labels results as moderation support and describes the prediction as a prompt for human inspection. A future study should test false positives and false negatives with reviewers from the intended environment.

13. Conclusion

ToxiGuard delivers a working binary toxic-comment classifier and Streamlit dashboard. In the reproduced run, six models were compared on the same 6,000-comment test set, and Logistic Regression ranked first by toxic-class F1. Its test results were 93.23% accuracy, 61.02% toxic precision, 81.39% toxic recall, 69.75% toxic F1, and 0.9576 ROC-AUC.

The project demonstrates data preparation, TF-IDF feature extraction, measured model selection, artifact serialization, and application deployment. Its outputs can help prioritize review, while human judgment remains necessary because the model makes both false positive and false negative errors.

14. Future Scope

- Collect feedback from human reviewers and tune the classification threshold for the intended moderation workload.
- Evaluate on comments from other platforms and languages before broader use.
- Add context-aware models and compare them with the current transparent linear baseline.
- Extend the interface to Jigsaw's separate toxicity categories after multi-label evaluation.
- Monitor model drift and document performance across different groups and language patterns.

REFERENCES

- [1] Jigsaw and Conversation AI, Toxic Comment Classification Challenge, <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge> (accessed 29 September 2026).
- [2] scikit-learn Developers, Text feature extraction and classification documentation, <https://scikit-learn.org/stable/> (accessed 29 September 2026).
- [3] Streamlit, Streamlit documentation, <https://docs.streamlit.io/> (accessed 29 September 2026).
- [4] Python Software Foundation, Python documentation, <https://docs.python.org/3/> (accessed 29 September 2026).
- [5] ToxiGuard project repository, `train_model.py`, `app.py`, source data, and generated outputs, local project files examined 29 September 2026.

Appendix A — Source Code

The following images show selected parts of the actual source files. The complete editable code remains in the ToxiGuard project directory.

Code Listing A.1: Shared Text-Cleaning Function

src/text_utils.py Text cleaning

```
24 def clean_text(value: object) -> str:
25     """Return the lightweight text representation used by the TF-IDF model.
26
27     The function deliberately does not remove common words or negation. In
28     short comments, words such as "not" can change meaning and are useful
29     signal for a traditional text classifier.
30     """
31     if not isinstance(value, str):
32         return ""
33
34     text = value.lower()
35     text = re.sub(r"https?://\S+|www\.\S+", " ", text)
36     text = re.sub(r"\[.*?\]", " ", text)
37     text = re.sub(r"[^a-z0-9'!?\\s]", " ", text)
38     return re.sub(r"\s+", " ", text).strip()
39
```

The `clean_text` function lowercases comments, removes URLs and selected markup, replaces unsupported characters, and normalizes whitespace. The same helper is imported during training and application inference.

Code Listing A.2: Feature Extraction and Model Candidates

train_model.py TF-IDF and candidates

```
139     sampled = stratified_sample(dataset)
140     X_train, X_test, y_train, y_test = train_test_split(
141         sampled["clean_text"], sampled["toxic"], test_size=0.20,
142         stratify=sampled["toxic"], random_state=RANDOM_STATE,
143     )
144     vectorizer = TfidfVectorizer(max_features=15000, ngram_range=(1, 2),
145                                 min_df=2, sublinear_tf=True)
146     X_train_tfidf = vectorizer.fit_transform(X_train)
147     X_test_tfidf = vectorizer.transform(X_test)
148     candidates = {
149         "Multinomial Naive Bayes": MultinomialNB(alpha=0.5),
150         "Complement Naive Bayes": ComplementNB(alpha=0.5),
151         "Logistic Regression": LogisticRegression(max_iter=1000,
152             class_weight="balanced", random_state=RANDOM_STATE),
153         "Linear SVM (Calibrated)": CalibratedClassifierCV(
154             LinearSVC(class_weight="balanced", random_state=RANDOM_STATE),
155             method="sigmoid", cv=3,
156         ),
157         "SGD Classifier": SGDClassifier(loss="modified_huber",
158             class_weight="balanced", max_iter=1500, random_state=RANDOM_STATE),
159         "Random Forest": RandomForestClassifier(
160             n_estimators=150, max_features="sqrt", min_samples_leaf=2,
161             class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE,
162         ),
163     }
```

The training script uses a stratified split and fits the TF-IDF vectorizer only on the training subset. The candidate dictionary defines the six classifiers compared in the experiment. Long source lines wrap visually in this listing.

Code Listing A.3: Model Selection and Serialization

train_model.py Selection and export

```
178     results = pd.DataFrame(rows).sort_values(  
179         ["F1 Score", "Recall", "Accuracy"], ascending=False  
180     ).reset_index(drop=True)  
181     results.insert(0, "Rank", np.arange(1, len(results) + 1))  
182     results.to_csv(OUTPUTS / "model_comparison.csv", index=False)  
183     results.to_csv(OUTPUTS / "model_comparison_results.csv", index=False)  
184     best_name = results.loc[0, "Model"]  
185     best_model = fitted_models[best_name]  
186     best_pred = predictions[best_name]  
187     best_probabilities = probabilities_by_model[best_name]  
188     joblib.dump(best_model, MODELS / "toxiguard_model.pkl")  
189     joblib.dump(vectorizer, MODELS / "tfidf_vectorizer.pkl")  
190
```

The results are ranked by toxic F1, recall, and accuracy. The script exports the comparison metrics and saves the selected classifier and TF-IDF vectorizer with joblib.

Code Listing A.4: Loading Application Artifacts

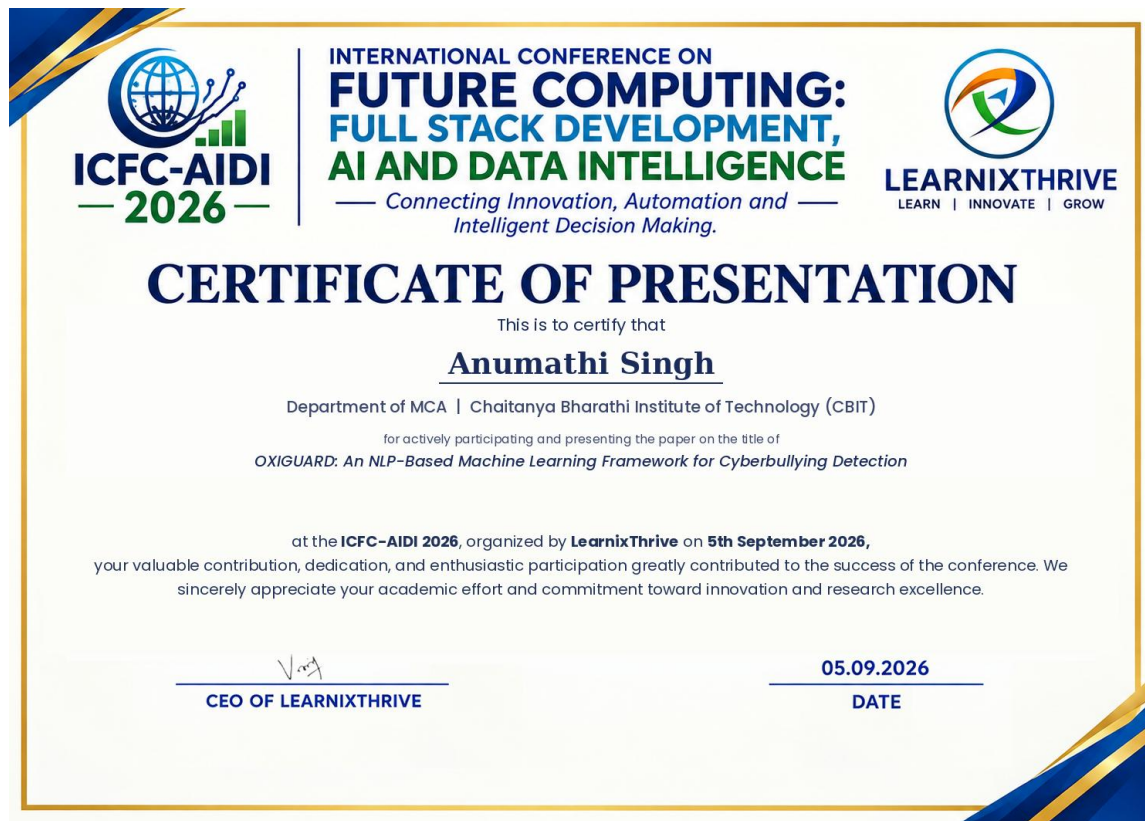
app.py Loading saved artifacts

```
177 @st.cache_resource(show_spinner=False)
178 def load_project_assets() -> tuple[object, object, dict, pd.DataFrame,
    pd.DataFrame, pd.DataFrame]:
179     """Load only artifacts produced by the canonical training script."""
180     required = {
181         "model": MODELS / "toxiguard_model.pkl",
182         "vectorizer": MODELS / "tfidf_vectorizer.pkl",
183         "metadata": OUTPUTS / "project_metadata.json",
184         "comparison": OUTPUTS / "model_comparison_results.csv",
185         "report": OUTPUTS / "classification_report.csv",
186         "top_words": OUTPUTS / "top_toxic_words.csv",
187     }
188     missing = [name for name, path in required.items() if not path.exists()]
189     if missing:
190         raise FileNotFoundError(", ".join(missing))
191
192     model = joblib.load(required["model"])
193     vectorizer = joblib.load(required["vectorizer"])
194     metadata = json.loads(required["metadata"].read_text(encoding="utf-8"))
195     comparison = pd.read_csv(required["comparison"]).sort_values("Rank").reset_index(drop=True)
196     report = pd.read_csv(required["report"], index_col=0)
197     top_words = pd.read_csv(required["top_words"])
198     return model, vectorizer, metadata, comparison, report, top_words
199
```

The cached loading function checks for the model, vectorizer, metadata, and evaluation files. It loads these artifacts for reuse by the Streamlit interface, avoiding retraining during a user session.

Appendix B — Paper Presentation Certificate

The presentation certificate is reproduced as issued. It records participation at ICFC-AIDI 2026 on 5 September 2026.



Appendix C — Project Outputs

The files below are the direct outputs of the reproduced training run. They can be inspected in the ToxiGuard project directory.

Table C.1: Project Output Files

File	Purpose
project_metadata.json	Run settings and selected-model summary
model_comparison.csv	Six-model held-out comparison
classification_report.csv	Per-class precision, recall, F1, and support
predictions_output.csv	Row-level test predictions and probabilities
eda_plots.png	Dataset exploration figures
model_comparison.png	Model performance chart
confusion_matrix.png	Errors for the selected classifier

The output files are in D:\ToxiGuard\outputs. The saved classifier and TF-IDF vectorizer are in D:\ToxiGuard\models.

Recorded training command: `python train_model.py`

Dashboard command: `python -m streamlit run app.py`